S2S-02: Step 2 — Strategize: Define Your Migration Approach

> **Series:** S2S — SaaS to SaaS Migration | **Notebook:** 2 of 9 | **Phase:** Plan | **Step:** Strategize | **Created:** March 2026 | **Last Updated:** 08/04/2026

With your discovery complete, it's time to turn inventory into action. This notebook helps you select a migration approach, sequence your operations, assess risks, and build a timeline that earns stakeholder confidence.

> **Migration Journey — You Are Here**
>
> **Plan:** 1. Discover → **2. Strategize** → 3. Design | **Upgrade:** 4. Prepare → 5. Execute → 6. Integrate | **Run:** 7. Expand → 8. Enable → 9. Optimize

---

Table of Contents

Prerequisites

Requirement	Description
Step 1 Complete	Discovery inventory from S2S-01 (entity counts, configuration scope, integration inventory, IAM assessment)
Stakeholder Access	Ability to engage infrastructure, security, and application teams
Dynatrace Account Team	Contact established for licensing and migration support
Risk Tolerance Understood	Organizational appetite for parallel operation duration and cost

1. The Three-Phase Framework

The SaaS-to-SaaS migration follows the same proven three-phase, nine-step framework used for Managed-to-SaaS. Each phase builds on the previous one, and skipping steps leads to preventable failures.

Phase Overview

Phase	Steps	Focus
Plan	1. Discover → 2. Strategize → 3. Design	Understand what you have, decide how to move it, design the target
Upgrade	4. Prepare → 5. Execute → 6. Integrate	Provision target, migrate agents and config, reconnect integrations
Run	7. Expand → 8. Enable → 9. Optimize	Extend coverage, activate new capabilities, tune for value

What Each Step Delivers

Step	Name	Key Deliverable
1	**Discover**	Complete environment inventory (entities, configs, integrations, IAM)

Step	Name	Key Deliverable
2	**Strategize**	Migration approach, timeline, risk assessment, success criteria
3	**Design**	Target SaaS architecture, network topology, IAM model
4	**Prepare**	Target tenant provisioned, ActiveGates deployed, network validated
5	**Execute**	Configurations migrated, OneAgents redirected, data flowing
6	**Integrate**	Webhooks, CI/CD, ITSM, and third-party tools reconnected
7	**Expand**	Additional environments, cloud integrations, OpenTelemetry
8	**Enable**	Workflows, OpenPipeline, Business Analytics, Notebooks activated
9	**Optimize**	Alert tuning, cost management, query performance, feature adoption

Time Investment by Phase

Phase	Typical Duration	Effort Level
Plan	2-4 weeks	Medium — analysis and decision-making
Upgrade	2-6 weeks	High — technical execution
Run	2-4 weeks	Medium — enablement and tuning

> **S2S vs. M2S:** The framework is identical, but SaaS-to-SaaS migrations typically move faster because there is no architecture upgrade (both tenants are Gen3 Grail). However, the absence of a SaaS Upgrade Assistant means you rely entirely on Monaco, Terraform, and the Settings API.

2. Migration Approach Selection

The single most consequential decision in your migration is the approach. Choose based on environment size, risk tolerance, and operational constraints.

Three Approaches

Approach	When to Use	Pros	Cons
Big Bang	< 500 hosts, simple environment	Fast, decisive, clean cutover	Higher risk, requires extensive prep
Phased by Environment	500-2,000 hosts	Lower risk, lessons learned per wave	Longer timeline, dual-run costs
Phased by Region/App	> 2,000 hosts or complex integrations	Lowest risk, maximum flexibility	Longest timeline, complex coordination

Big Bang Migration

All agents and configurations migrate in a single maintenance window.

Factor	Detail
Typical window	4-8 hours
Best for	Smaller environments with flexible maintenance windows
Requires	Complete preparation, tested rollback procedure
Risk	Higher — all-or-nothing in a single window

Phased by Environment

Migrate in waves: Dev → Staging → Production.

Factor	Detail
Typical duration	3-6 weeks (1-2 weeks per wave)
Best for	Mid-size environments with distinct environment tiers
Requires	Parallel operation budget, wave-by-wave validation
Risk	Medium — each wave validates the next

Phased by Region or Application

Migrate by geography (EMEA → APAC → Americas) or by application criticality.

Factor	Detail
Typical duration	6-12 weeks
Best for	Large, complex environments with regional or app-team ownership
Requires	Complex coordination, extended parallel operation
Risk	Lowest — isolated blast radius per wave

Multi-Source Consolidation (Sequential Pattern)

When consolidating multiple source tenants into a single target, migrate sources **sequentially**, not in parallel. Complete one source before starting the next.

Factor	Detail
Pattern	Source 1 → validate → Source 2 → validate → ... → Source N
Why sequential	Isolates issues to a single source; avoids configuration collisions; simplifies rollback
Best for	Tenant consolidation (M&A, regional merge)
K8s complexity	If one source has Kubernetes and another does not, migrate the simpler source first — K8s requires DynaKube redeployment (not `oneagentctl`) and introduces additional validation steps
Typical duration	Add 2-3 weeks per additional source tenant

> **Lesson from real migrations:** A two-source consolidation (70 non-K8s hosts + 38 K8s hosts) used sequential migration — the simpler source first to validate the Monaco → SUA → target workflow, then the K8s source second. This isolated a DynaKube redeployment issue that would have been much harder to diagnose if both sources migrated simultaneously.

S2S Advantages Over M2S

SaaS-to-SaaS migration has inherent advantages that affect approach selection:

Advantage	Impact on Strategy
Both environments are SaaS	Easier parallel operation — no on-premises infrastructure to maintain
No architecture upgrade	Both tenants are Gen3 Grail — no cluster version concerns
Cloud-hosted infrastructure	Network connectivity is simpler (SaaS-to-SaaS, not on-prem-to-cloud)
Identical feature set	Source and target have the same capabilities — no feature gaps

S2S Risks vs. M2S

Risk Factor	S2S Impact
Entity IDs change completely	Every entity gets a new ID in the target tenant — dashboards, SLOs, and alerts with entity references must be remapped
No SaaS Upgrade Assistant	All configuration migration relies on Monaco, Terraform, or Settings API — no automated assistant
Rollback is harder	Unlike M2S where Managed can remain as fallback, reverting S2S means redirecting agents back to a tenant that has data gaps
Double licensing cost	Both tenants consume DPS during parallel operation — budget accordingly

Decision Matrix

Factor	Big Bang	Phased by Env	Phased by Region/App
Hosts < 500	Recommended	Optional	Unnecessary
Hosts 500-2,000	Possible	Recommended	Optional
Hosts > 2,000	Not recommended	Possible	Recommended
Simple integrations	Recommended	Optional	Unnecessary
Complex integrations	Risky	Recommended	Recommended

Factor	Big Bang	Phased by Env	Phased by Region/App
Short timeline required	Fastest	Moderate	Longest
Risk-averse organization	Higher risk	Lower risk	Lowest risk
Multi-source consolidation	Not recommended	Sequential by source	Sequential by source

3. S2S Order of Operations

Regardless of which approach you choose, the migration must follow this exact 11-step sequence. Steps cannot be reordered — each depends on the previous one completing successfully. The rightmost column shows which notebook covers each operation in detail.

#	Operation	Why This Order	Covered In
1	**Assess** — Inventory source tenant	Cannot plan what you don't know	Step 4: Prepare
2	**Provision** — Target SaaS tenant and access (SSO/IAM)	Target must exist before anything moves	Step 4: Prepare
3	**Export** — Configuration from source tenant	Need config before import	Step 4: Prepare
4	**Migrate** — Configuration to target tenant	Settings must be in place before agents report	Step 5: Execute
5	**Rebuild** — Dashboards, SLOs, alerts	Operational visibility before cutover	Step 5: Execute
6	**Redirect** — OneAgents and operators to target	Agents begin reporting to target tenant	Step 5: Execute
7	**Reconnect** — Cloud integrations and extensions	Restore external connections in target	Step 6: Integrate
8	**Migrate** — Any remaining configuration	Entity-dependent configs that require agent data	Step 6: Integrate

#	Operation	Why This Order	Covered In
9	**Validate** — Data flow and performance	Confirm everything works before declaring success	Step 9: Optimize
10	**Cutover** — Full switch to target tenant	Formal declaration that target is primary	Step 9: Optimize
11	**Decommission** — Source tenant	Only after validation period complete	Step 9: Optimize

> **Key Difference from M2S:** In M2S, the SaaS Upgrade Assistant handles Steps 3-5 semi-automatically. In S2S, you use `monaco download` / `monaco deploy` (or Terraform) for export and import. The sequence is the same, but the tooling is manual.

Critical Dependencies

Dependency	Impact if Missed
Configuration before agent redirect	No alerting, no dashboards during cutover
SSO/IAM before user access	Teams locked out of new target tenant
ActiveGates before OneAgents	Agents have nowhere to route — data loss
Firewall rules before anything	All agent traffic blocked — complete failure
Entity data before SLO/dashboard migration	Entity-referenced configs will fail without entity IDs in target

4. Migration Considerations

These are the factors specific to SaaS-to-SaaS migration that are often overlooked.

Entity ID Changes

This is the single biggest difference between S2S and M2S. In a Managed-to-SaaS migration, some entity IDs can persist via `oneagentctl`. In SaaS-to-SaaS, **every entity gets a new ID**.

Impact Area	What Changes	Mitigation
Dashboards	Entity filters and tile queries reference source IDs	Export, remap IDs using entity name matching, reimport
SLO definitions	Metric expressions with entity selectors	Recreate with target entity IDs after agents report
Notification rules	Entity-based alert filters	Update selectors after entity discovery in target
Management zones	Entity-based rules (if using legacy MZ)	Rules based on tags/properties migrate cleanly; entity-ID rules need remapping
Automation workflows	Hardcoded entity IDs in workflow actions	Parameterize workflows to use names/tags instead of IDs

> **Best Practice:** Before migration, refactor any configuration that references entity IDs to use tags, naming patterns, or properties instead. This makes migration significantly simpler.

Dynatrace Intelligence Baselines

Dynatrace Intelligence learns normal behavior from historical data. In a new tenant, it starts from scratch.

Baseline Type	Time to Establish	Notes
Availability	2-3 days	Binary metric, fast learning
Response time	1-2 weeks	Requires weekday/weekend patterns
Error rate	1-2 weeks	Needs regular traffic patterns
Resource utilization	2-4 weeks	Needs full business cycle

Parallel Operation

Both tenants are cloud-hosted, which makes parallel operation easier than M2S but introduces cost considerations.

Consideration	Detail
Double licensing	Both tenants consume DPS during parallel period — coordinate with Dynatrace account team
Duration	2-4 weeks minimum for Dynatrace Intelligence baselines; longer for complex environments
Agent dual-reporting	OneAgent cannot report to two tenants — use phased waves, not dual-send
Cloud integrations	Can point at both tenants simultaneously (separate credentials)

Items That Cannot Be Exported

Item	Reason	Workaround
Credential Vault secrets	Secrets cannot be exported from any tenant	Recreate each credential manually in target
Cloud integration credentials	AWS/Azure/GCP keys are tenant-specific	Create new integration credentials in target
Synthetic private locations	Tied to source ActiveGates	Recreate with target tenant ActiveGates
API tokens	Secrets, cannot be exported	Create new tokens in target
OAuth client secrets	Secrets, cannot be exported	Create new OAuth clients in target
Historical data	Stored in source tenant Grail	Accept data gap or extend parallel period
Dynatrace Intelligence baselines	Learned from source data	Allow 2-4 weeks to retrain in target
Problem history	Stored in source tenant	Export key problems as documentation
Extensions 2.0	Neither Monaco nor Terraform supports Extensions 2.0	Manual reinstall from Dynatrace Hub

Use Your Discovery Data to Assess Dependencies

The queries from Step 1 (Discover) provide the data you need to assess these considerations. If you haven't run them yet, go back to **S2S-01** and complete the discovery first.

Once your agents are reporting to the target tenant, use these DQL queries to validate coverage and identify gaps.

```
// Count monitored entities by type — compare against discovery inventory
fetch dt.entity.host
| summarize hosts = count()
| append [fetch dt.entity.service | summarize services = count()]
| append [fetch dt.entity.application | summarize applications = count()]
| append [fetch dt.entity.process_group | summarize process_groups = count()]

// Keep classic: this is a completeness inventory spanning types not all
present on Grail
// Smartscape (e.g. application, process groups). Smartscape would count live
topology, not
// the full monitored inventory — the wrong basis for a migration-completeness
check.
```

```
// Verify ActiveGate connectivity in target tenant — all Environment
ActiveGates should be reporting
smartscapeNodes "ACTIVEGATE"
| fields name, zone = dt.network_zone.id, group = dt.active_gate.group.name
| sort name asc

// Correction (verified 07/2026): this cell previously ran `fetch
dt.entity.active_gate` and
// carried a note claiming Smartscape had no ActiveGate node. Both were wrong.
There is NO classic
// ActiveGate entity type in any spelling (active_gate,
environment_active_gate,
// environment_activegate), so the classic query returned zero rows in every
tenant —
// indistinguishable from "no ActiveGates deployed". `smartscapeNodes
"ACTIVEGATE"` (no
// underscore) is the working path, and it works on tenants today.
// Field maps: entity.name → name, softwareVersion → dt.active_gate.version,
// networkZone → dt.network_zone.id.
```

```
// Check OneAgent version distribution — identify agents that may need
upgrading
fetch dt.entity.host
| fieldsAdd version = installerVersion
| summarize hostCount = count(), by:{version}
| sort hostCount desc

// No Smartscape equivalent: installerVersion (OneAgent version) is not a
Smartscape node
// field, so this version distribution stays on the classic entity store.
```

5. Risk Assessment

Every migration carries risk. The goal is not to eliminate risk but to identify, quantify, and mitigate it before execution begins.

Risk Register

Risk	Impact	Likelihood	Mitigation
Entity ID remapping failures	High	High	Refactor configs to use tags/names before migration; validate entity references post-import
Data gaps during cutover	High	Medium	Use phased approach; minimize per-wave gap to < 15 minutes
Dynatrace Intelligence false positives	Medium	High	Communicate baseline learning period to on-call teams; suppress non-critical alerts for 2-4 weeks
Configuration drift	Medium	Medium	Freeze source tenant changes during migration; use Monaco manifest for consistent exports
Integration failures	High	Medium	Test all webhooks and APIs in target before cutover; verify endpoints and tokens
Rollback complexity	High	Low	Document rollback procedure; keep source tenant active during entire parallel period

Risk	Impact	Likelihood	Mitigation
Licensing overlap costs	Medium	High	Coordinate with Dynatrace account team early; negotiate parallel operation terms
SSO/SAML misconfiguration	High	Medium	Test IdP integration before any user access; verify SAML message signing
Extensions 2.0 gaps	Medium	Medium	Inventory extensions in source; verify availability in Dynatrace Hub before migration
Credential Vault recreation	Medium	High	Inventory all credentials early; coordinate with security team for secret provisioning

Risk Scoring

Impact Level	Definition
High	Migration failure, data loss, or monitoring gap > 1 hour
Medium	Partial functionality loss or delayed timeline
Low	Inconvenience or minor cost impact

> **Tip:** Assign an owner to each risk. Unowned risks are unmitigated risks.

6. Defining Success Criteria

Define measurable success criteria before migration starts. These criteria determine when the migration is "done" and when you can decommission the source tenant.

Target Metrics

Metric	Target	How to Measure
Host coverage	100% of discovered hosts reporting	Compare DQL host count in target to discovery inventory

Metric	Target	How to Measure
Service discovery	100% of known services detected	Compare DQL service count to discovery inventory
Data gaps	< 15 minutes per wave during cutover	Review data availability in target after each wave
Alert delivery	100% of critical alerts firing correctly	Trigger test alerts; verify notification channels
Dashboard availability	100% of migrated dashboards rendering	Spot-check each dashboard in target; verify entity ID remapping
Integration success	100% of external integrations operational	Test each webhook, API, and ITSM connection
User access	All users can authenticate via SSO	Verify SSO login for each role/group
SLO accuracy	All SLOs evaluating correctly	Confirm metric expressions resolve with target entity IDs
Baseline established	Dynatrace Intelligence baseline period complete (2-4 weeks)	Confirm Dynatrace Intelligence is generating problems correctly — no excessive false positives

```
// Post-migration validation — compare host count to your discovery inventory
fetch dt.entity.host
| summarize totalHosts = count()
| fieldsAdd target = "&lt;YOUR_DISCOVERY_COUNT&gt;", coverage = "Compare
totalHosts to target"

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "HOST"
//   | summarize totalHosts = count()
//   | fieldsAdd target = "&lt;YOUR_DISCOVERY_COUNT&gt;", coverage = "Compare
totalHosts to target"
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
```

```
// Post-migration validation — check for recent detected problems (confirms AI baseline is building)
fetch dt.davis.problems, from:-24h
| summarize problemCount = count(), by:{event.status}
| sort problemCount desc
```

7. Timeline Planning

Build your timeline working backward from the desired source tenant decommission date.

Milestone Durations

Milestone	Duration	Notes
Plan (Steps 1-3)	2-4 weeks	Front-load planning — rushed planning causes failed execution
Prepare (Step 4)	1-2 weeks	Target tenant provisioning, ActiveGate deployment, network validation
Execute (Step 5)	1-3 weeks	Config migration + OneAgent redirect (per wave for phased)
Integrate (Step 6)	1-2 weeks	Cloud integrations, webhooks, ITSM reconnection
Parallel operation	2-4 weeks	Both tenants running — critical for Dynatrace Intelligence baselines
Run (Steps 7-9)	2-4 weeks	Expand coverage, enable new features, optimize
Total	**4-12 weeks**	Depends on environment size and approach

Sample Timelines by Approach

Approach	Week 1-2	Week 3-4	Week 5-6	Week 7-8	Week 9-12
Big Bang	Plan + Prepare	Execute + Validate	Parallel Run	Decommission	—

Approach	Week 1-2	Week 3-4	Week 5-6	Week 7-8	Week 9-12
Phased by Env	Plan + Prepare	Dev wave	Staging wave	Prod wave	Parallel + Decommission
Phased by Region	Plan + Prepare	Region 1	Region 2	Region 3	Parallel + Decommission

> **Important:** The parallel operation period is non-negotiable. Dynatrace Intelligence needs 2-4 weeks to build baselines in the target tenant before you can trust its problem detection. Do not decommission the source tenant until baselines are established.

Migration Plan Checklist

When building your high-level migration plan, ensure these items are addressed:

1. **Catalog and evaluate** monitoring components (OneAgents, ActiveGates, extensions, external sources) — any version updates needed?
2. **Identify** configurations and settings — what migrates via Monaco/Terraform vs. what requires manual effort?
3. **Leave legacy behind** — excessive or outdated configuration items can be left behind; apply naming standards in the target
4. **Understand procedures and timeline** for ActiveGate, OneAgent, and configuration migrations plus application restarts
5. **Minimize dual running** — provide a seamless cutover for Dynatrace users
   - What can be done over a weekend or outside of peak/business hours?
   - Group components into migration waves; learn from non-production first

Things to Include in Your Plan

Item	Detail
SSO implementation	IdP configuration, DNS domain validation, group mapping
Email allowlisting	Dynatrace email domain for SaaS platform/problem/security notifications

Item	Detail
ActiveGate provisioning	New Environment ActiveGates for target tenant
Network change requests	Connectivity between agents, ActiveGates, and target SaaS tenant
Integration connectivity	External on-premise or SaaS systems, Dynatrace API, webhook integrations
Credential recreation	Credential Vault entries, OAuth clients, API tokens in target
Deployment automation	Prepare OneAgent and ActiveGate redeployment scripts (Ansible, Puppet, Terraform, PowerShell)
Training	Train team on Monaco/Terraform migration tooling and target tenant administration
Execution plan	Environments and applications — in what order and what waves

8. The 90/10 Rule

Based on successful SaaS-to-SaaS migrations, a consistent pattern emerges:

> **90% of configurations migrate automatically** via Monaco or Terraform. The remaining **10% takes 90% of the manual effort**.

What Migrates Automatically (the 90%)

Category	Tool
Settings 2.0 (all schemas)	Monaco `monaco download` / `monaco deploy`
Dashboards and Notebooks	Monaco (documents type)
SLO definitions	Monaco (settings type)
Notification rules	Monaco (settings type)
Workflows and automations	Monaco (automations type)

Category	Tool
OpenPipeline processing rules	Monaco (openpipeline type)
Grail bucket definitions	Monaco (bucket type)
Segments	Monaco (segment type)
Synthetic monitors	Monaco (requires classic API token)
IAM policies and groups	Terraform (only tool that supports IAM)

What Requires Manual Effort (the 10%)

Item	Why Manual	Effort Level
Entity ID remapping	IDs are unique per tenant — all references must be updated	High — most time-consuming item
Credential Vault entries	Secrets cannot be exported from any tenant	High — must recreate each credential
Cloud integration credentials	AWS/Azure/GCP keys are tenant-specific	Medium — reconfigure each integration
Synthetic private locations	Tied to source ActiveGates	Medium — recreate with target ActiveGates
Extensions 2.0	Neither Monaco nor Terraform supports Extensions 2.0	Medium — manual reinstall from Hub
Problem notification webhooks	URLs and tokens may change between environments	Medium — update each endpoint
Custom scripts and automation	Hardcoded source tenant URLs and tokens	Medium — update all references
Third-party ITSM integrations	Webhook endpoints change	Medium — reconfigure each connection

Budget Accordingly

Planning Item	Recommendation
Identify the 10% early	Use your discovery inventory to list every manual item
Assign owners	Each manual item needs a person responsible
Track separately	Manual items need their own checklist — do not mix with automated migration
Test before cutover	Every manual item should be validated in target before redirecting agents

9. Step Completion Checklist

Do not proceed to Step 3 (Design) until all items are confirmed.

Checkpoint	Status
Migration approach selected (Big Bang / Phased by Env / Phased by Region)	[ ]
Order of operations documented and reviewed with stakeholders	[ ]
Entity ID remapping strategy defined (tags vs. names vs. post-migration scripting)	[ ]
Risk assessment completed with owners assigned to each risk	[ ]
Success criteria defined with measurable targets	[ ]
High-level timeline established with milestone dates	[ ]
Dynatrace account team engaged for licensing and parallel operation support	[ ]
90/10 manual effort items identified and owners assigned	[ ]
Communication plan drafted for affected teams	[ ]
Rollback procedure documented	[ ]

Next Step

> **S2S-03: Step 3 — Design** — Create your target SaaS architecture, including network topology, ActiveGate placement, IAM model, and configuration mapping.

Summary

In this notebook, you:

- Reviewed the three-phase, nine-step migration framework adapted for SaaS-to-SaaS
- Selected a migration approach based on your environment size and risk tolerance
- Documented the 11-step order of operations and critical dependencies
- Assessed S2S-specific considerations including entity ID changes, Dynatrace Intelligence baselines, and parallel operation costs
- Completed a risk register with S2S-specific mitigations and owners
- Defined measurable success criteria for migration completion
- Built a timeline with milestone durations
- Identified the 90/10 split between automated (Monaco/Terraform) and manual migration effort

> **Key Takeaway:** SaaS-to-SaaS migration is simpler than Managed-to-SaaS in architecture but harder in entity management. The absence of a SaaS Upgrade Assistant means you own the entire toolchain. A clear approach, documented risks, and measurable success criteria are what separate migrations that succeed from those that spiral into firefighting.

Continue to S2S-03: Step 3 — Design to create your target SaaS architecture.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.